

ARK-456 — Fail-Closed Dependency Loss

PREREGISTRATION (committed before execution)

Experiment ID: ARK-456

Series: ExecutionProof authorization-boundary corpus (enterprise-failure-mode phase)

Substrate: Classical software (no quantum hardware, no cryptography)

Question: When required external dependencies are unavailable or corrupted, does the system fail closed (HOLD or DENY) rather than silently ALLOW?

1. Motivation

Enterprise authorization systems depend on external services: identity providers, policy registries, evidence schemas, time sources, signing keys, audit stores. When these dependencies degrade — network partitions, service outages, configuration corruption, key rotation failures — the system faces a choice:

1. **Fail open** — optimistically ALLOW requests despite missing data (dangerous)
2. **Fail closed** — refuse to operate without required dependencies (safe)

Real-world incidents often stem from degraded infrastructure, not malicious actors. A system that silently allows unauthorized actions when a dependency is unavailable violates the fail-safe principle.

This experiment tests: Does an authorization guard refuse to ALLOW when critical dependencies are missing, or does it degrade gracefully to HOLD/DENY?

2. Design

Structure

- **8 arms** × 100 scenarios per arm = **800 evaluation decisions**
- Each arm represents a specific dependency-loss condition
- Dual independent guards (V1 JavaScript, V2 Python) evaluate each scenario

Dependency Model

Each authorization scenario carries a `dependency_state` describing the availability of six critical services:

Dependency	Role	States
identity_provider	Resolves subject identity and attributes	available / unavailable / corrupted
policy_registry	Provides authorization policies	available / unavailable / corrupted
evidence_schema	Validates evidence structure	available / unavailable / corrupted
clock_source	Provides trusted time	available / unavailable / corrupted
signature_key	Signs authorization records	available / unavailable / corrupted
audit_store	Records authorization events	available / unavailable / corrupted

Authorization Request

Each scenario includes a baseline authorization request:

```
{
  "action": "transfer" || "approve" || "delegate" || "execute",
  "subject": "agent-X",
  "resource": "wallet-Y",
  "context": { "amount": ..., "destination": ..., ... }
}
```

In a **healthy system (arm 1)**, the request is evaluated normally and may ALLOW if valid.

In a **degraded system (arms 2-8)**, one or more dependencies are unavailable/corrupted.

Decision Procedure (Both Guards)

```
evaluate(request, dependency_state):
  1. Check required dependencies:
    - identity_provider must be "available"
    - policy_registry must be "available"
    - evidence_schema must be "available"
    - clock_source must be "available"
    - signature_key must be "available"
    - audit_store must be "available"

  2. If ANY required dependency is NOT "available":
    → return DENY (fail-closed: cannot verify authorization without required data)

  3. If ALL dependencies available:
    → evaluate authorization normally
    → return ALLOW if valid, HOLD if borderline, DENY if invalid
```

Key property: A missing or corrupted dependency **immediately triggers DENY**, regardless of the authorization request's inherent validity.

3. Arms

Arm	Condition	Degraded Dependency	Expected
1	BASELINE-healthy	None (all available)	ALLOW (if request valid)
2	IDENTITY-provider-unavailable	identity_provider = "unavailable"	DENY
3	POLICY-registry-unavailable	policy_registry = "unavailable"	DENY
4	EVIDENCE-schema-corrupted	evidence_schema = "corrupted"	DENY
5	CLOCK-source-unreachable	clock_source = "unavailable"	DENY
6	SIGNATURE-key-missing	signature_key = "unavailable"	DENY
7	AUDIT-store-unreachable	audit_store = "unavailable"	DENY
8	MULTIPLE-deps-degraded	3+ dependencies unavailable/corrupted	DENY

- **Arm 1** (baseline): All dependencies healthy; authorization request is valid → expected **ALLOW**
- **Arms 2-8** (degraded): At least one required dependency unavailable/corrupted → expected **DENY** (fail-closed), never ALLOW

4. Pass Criteria (Preregistered)

Let:

- **g_allow_baseline** = ALLOW rate in Arm 1 (baseline-healthy)
- **g_deny_degraded_min** = minimum DENY rate across Arms 2-8 (degraded conditions)
- **g_margin** = $\min(g_allow_baseline, g_deny_degraded_min) - 0.05$

Criteria:

- **C1:** $g_allow_baseline \geq 0.95$
(Healthy system allows valid requests)

- **C2:** $g_deny_degraded_min \geq 0.95$
(Degraded system refuses to operate — fail-closed)
- **C3:** $g_margin \geq 0.90$
(Strong separation between healthy-allow and degraded-deny)

Verdict: PASS if $C1 \wedge C2 \wedge C3$, else FAIL. Recorded verdict stands as executed.

5. Dependency-Loss Effectiveness Gate

Before evaluating any arm, a **structural oracle** verifies that each scenario genuinely encodes its intended dependency condition. For example:

- Arm 2 scenarios must have `identity_provider = "unavailable"` and all other dependencies `"available"`
- Arm 8 scenarios must have ≥ 3 dependencies not `"available"`

If any scenario in an arm fails the effectiveness check, the arm is **aborted** — the experiment does not proceed with inert test cases. This gate prevents ARK-455-style no-op defects where a scenario appears to test a condition but does not actually encode it.

Abort condition: If any scenario in any arm is not dependency-loss-effective, the arm run is aborted and the experiment is stopped.

6. Dual Independent Guards

V1 Guard (JavaScript, no dependencies)

- Pure logic, no external libraries
- Reads `dependency_state` and applies the decision procedure
- Outputs: `{decision: "ALLOW" | "HOLD" | "DENY", reason: "..."}"`

V2 Guard (Python)

- Independent implementation of the same procedure
- Reads `dependency_state` and applies the decision procedure
- Outputs: `{decision: "ALLOW" | "HOLD" | "DENY", reason: "..."}"`

Concordance requirement: V1 and V2 must agree on every decision. Disagreement indicates ambiguity in the procedure definition.

7. Kill-Gate Calibration

Before running the 8 arms, generate 100 calibration scenarios (mixed across all 8 arms) and evaluate with both guards. Verify:

1. **Dependency-loss effectiveness:** All 100 scenarios pass the structural oracle
2. **V1-V2 concordance:** $\geq 99\%$ agreement

If the kill-gate fails, the experiment is **aborted** — the dual-guard setup is not ready.

8. Execution Procedure

1. **LOCK** — Commit this preregistration, dual guards, generator, and a SHA-256 MANIFEST **before** any scenario is generated or evaluated
 2. **Kill-gate** — Generate 100 calibration scenarios; check effectiveness and concordance
 3. **Arm execution** — For each arm 1-8:
 - Generate 100 scenarios (deterministic seed)
 - Verify dependency-loss effectiveness (abort if any scenario fails)
 - Evaluate with V1 guard
 - Evaluate with V2 guard
 - Record concordance
 4. **Compute overall metrics** — `g_allow_baseline`, `g_deny_degraded_min`, `g_margin`
 5. **Apply criteria** — C1, C2, C3 → VERDICT
 6. **Record results** — Fill MANIFEST OUTCOME; commit results; create tags
-

9. Constraints and Scope

What this tests:

- Whether a guard refuses to ALLOW when critical dependencies are unavailable
- Whether fail-closed behavior is correctly implemented
- Whether the decision procedure is clear enough for independent implementation

What this does NOT test:

- Real network partitions, service degradation timings, or retry logic
- Actual identity provider protocols, policy schema formats, or cryptographic key storage
- Recovery procedures after dependencies come back online
- Performance or scalability under partial outages
- Caching, fallback, or degraded-mode policies

Constraints: This is a classical/software boundary-logic testbed validating fail-closed behavior in isolation, not the end-to-end enterprise infrastructure stack.

10. Provenance and Integrity

- **LOCK before execution:** Preregistration + dual guards + generator + MANIFEST (SHA-256) committed before any scenario generated or evaluated
 - **No post-hoc criterion changes:** The three pass criteria (C1, C2, C3) are fixed at LOCK time
 - **Honest reporting:** If the system fails closed correctly, report PASS. If it silently allows under degraded conditions, report honest FAIL.
 - **Commits not cryptographically signed:** Provenance = commit history + MANIFEST SHA-256 hashes
-

11. Real-World Relevance

Dependency failures are routine in enterprise systems:

- Identity provider down during network partition
- Policy registry unreachable during cloud region outage
- Clock source drift or NTP failure
- Key rotation script fails, leaving signing key unavailable
- Audit store fills up or becomes read-only
- Multiple simultaneous failures during infrastructure incidents

A system that **silently allows** actions when it cannot verify authorization is dangerous. ARK-456 validates that the guard **refuses to operate** under degraded conditions.

12. Expected Outcome

If the guard is correctly fail-closed:

- **Arm 1 (healthy):** ALLOW rate $\approx$ 100% (valid requests proceed)
- **Arms 2-8 (degraded):** DENY rate $\approx$ 100% (missing dependencies $\rightarrow$ refuse to operate)
- **V1-V2 concordance:** 100% (decision procedure is unambiguous)
- **VERDICT:** PASS

If the guard fails open or degrades silently:

- **Arms 2-8:** ALLOW rate $>$ 0% (dangerous — allowing without verification)
 - **VERDICT:** FAIL (honest)
-

Investigator: Derek Hone, Remnant Fieldworks Inc.

Executor: Abacus.AI autonomous agent (supervised)

Series: ExecutionProof™ authorization-boundary corpus

Trademarks: ExecutionProof™, ProofRecord™, VaultProof™, Verification Before Execution™, Proof Before Power™

If it cannot be verified, it cannot execute.